
Computer Graphics Coursework report

Linlang Zou
5684282

University of Warwick

December 14, 2025

This project implements a complete 3D game engine based on DirectX 12, including core features such as texture lighting, mesh rendering, animation systems, collision detection, normal mapping, and particle systems. Through the GEM format for data-driven loading, efficient resource management and rendering pipelines are achieved.

1 3D Game Level with Textures and Lighting

The game level is composed of multiple 3D objects, including two types of static model trees randomly generated using fixed random numbers, a dinosaur with a simple state machine, a first-person perspective weapon, and a soldier synchronized with first-person actions as UI. The texture system is implemented through `Texture.h` and `Texture.cpp`, supporting multiple texture bindings. Lighting calculations are performed in the pixel shader using a simple directional light model.

```
1 float3 lightDir = normalize(float3(1.0, 1.0, 1.0));
2 float NdotL = max(dot(worldNormal, lightDir), 0.0);
3 colour.rgb *= (0.3 + 0.7 * NdotL);
```

2 Mesh Rendering System

Mesh rendering is implemented through `Mesh.h` and `Mesh.cpp`, supporting two types: static meshes and animated meshes. Static meshes contain position, normal, tangent, and texture coordinates, while animated

meshes additionally include bone IDs and weight information. Mesh data is stored through vertex buffers and index buffers, using DirectX 12's resource management mechanism.

```
1 struct STATIC_VERTEX {
2     Vec3 pos;
3     Vec3 normal;
4     Vec3 tangent;
5     float tu, tv;
6 };
7
8 struct ANIMATED_VERTEX {
9     Vec3 pos;
10    Vec3 normal;
11    Vec3 tangent;
12    float tu, tv;
13    unsigned int bonesIDs[4];
14    float boneWeights[4];
15 };
```

3 Custom File Format (GEM) Data-Driven Loading

The project implements a custom GEM binary file format, loaded through `GEMLoader.h`. This format supports static and animated models, including mesh data, material properties, bone information, and animation sequences. The loader reads through binary streams, efficiently converting model data into the engine's internal format.

```
1 void GEMModelLoader::load(std::string filename,
2     std::vector<GEMMesh>& meshes, GEMAnimation&
3     animation) {
4     std::ifstream file(filename, std::ios::
5         binary);
6     unsigned int n = 0;
7     file.read(reinterpret_cast<char*>(&n),
8         sizeof(unsigned int));
```

6 }

4 Opaque Mesh Rendering

Opaque meshes use standard depth testing and blend states, managed through pipeline state objects in `PSO.h`. Most game objects (such as trees, characters, and weapons) use opaque rendering, ensuring correct depth sorting and occlusion relationships.

5 Alpha Test Meshes

The leaf particle system uses alpha testing to achieve transparency effects. A Billboard-based particle system is implemented in `Particles.h` and `Particles.cpp`, where each particle is a quad facing the camera. The shader uses the `clip()` function for alpha testing, discarding pixels with transparency below the threshold.

```
1 float4 colour = tex.Sample(samplerLinear, input.
  TexCoords);
2 clip(colour.a - 0.01);
```

6 Vertex Shader Animation

The animation system is implemented through skeletal animation. The vertex shader `VSAnim.txt` receives an array of bone transformation matrices and performs blending calculations based on the vertex's bone IDs and weights. The animation controller `AnimationInstance` is responsible for updating bone transformations, supporting playback and switching of multiple animation sequences.

```
1 float4x4 transform;
2 transform = bones[input.BoneIDs[0]] * input.
  BoneWeights[0];
3 transform += bones[input.BoneIDs[1]] * input.
  BoneWeights[1];
4 transform += bones[input.BoneIDs[2]] * input.
  BoneWeights[2];
5 transform += bones[input.BoneIDs[3]] * input.
  BoneWeights[3];
6 output.Pos = mul(pos, transform);

1 void AnimationInstance::update(std::string name,
  float dt) {
2   if (name == usingAnimation) {
3     t += dt;
4   } else {
5     usingAnimation = name;
6     t = 0;
7   }
8   animation->calcFrame(name, t, frame,
  interpolationFact);
9   for (int i = 0; i < animation->bonesSize();
  i++) {
10    matrices[i] = animation->
  interpolateBoneToGlobal(
11      name, matrices, frame,
  interpolationFact, i);
12  }
13 }
```

7 Mesh Instancing (Trees)

Tree rendering uses instancing technology. In the `Tree` class, a world transformation matrix is generated for each tree and stored in the `worldMatrices` array. During rendering, all instances are traversed, updating the world matrix for each instance and drawing, greatly improving rendering efficiency.

```
1 void Tree::init(Core* core, PSOManager* psos,
  Shaders* shaders, int treeCount) {
2   generatePositions(treeCount, -70.0f, 70.0f,
  -70.0f, 70.0f);
3   worldMatrices.reserve(numInstances);
4   for (int i = 0; i < numInstances; i++) {
5     Matrix W = Matrix::scaling(scale) *
  Matrix::translation(positions[i]);
6     worldMatrices.push_back(W);
7   }
8 }
9
10 void Tree::draw(Core* core, PSOManager* psos,
  Shaders* shaders, Matrix& vp) {
11   for (int i = 0; i < numInstances; i++) {
12     model.updateWorld(shaders, worldMatrices
  [i]);
13     model.draw(core, psos, shaders, vp);
14   }
15 }
```

8 First/Third Person Camera Control

The camera system is implemented in `Camera.h` and `Camera.cpp`, supporting keyboard (WASD) movement and mouse view control. Mouse input controls yaw and pitch angles, while keyboard input controls forward, backward, left, and right movement. The camera also integrates collision detection with dinosaur entities, preventing it from passing through them.

```
1 void Camera::update(float dt, Window* window) {
2   float xOffset = (float)(centerX - window->
  mouseX) * mouseSensitivity;
3   float yOffset = (float)(centerY - window->
  mouseY) * mouseSensitivity;
4   yaw += xOffset;
5   pitch += yOffset;
6
7   Vec3 moveDir = cameraFront;
8   moveDir.y = 0;
9   moveDir = moveDir.normalize();
10  if (window->keys['W']) moveDelta = moveDelta
  + moveDir * speed;
11  if (window->keys['S']) moveDelta = moveDelta
  - moveDir * speed;
12  if (window->keys['A']) moveDelta = moveDelta
  - cameraRight * speed;
13  if (window->keys['D']) moveDelta = moveDelta
  + cameraRight * speed;
14
15  Vec3 newPos = cameraPos + moveDelta;
16  if (collisionBox != nullptr && collisionBox
  ->checkCollision(newPos)) {
17    return;
18  }
19  cameraPos = newPos;
20 }
```

9 Animated Character and Player Interaction

Animated characters in the game, such as the dinosaur TRexEntity and soldier SoldierEntity, manage multiple animation sequences (idle, movement, attack, etc.) through animation controllers. The dinosaur character always moves toward the camera facing the player until the player enters attack range, then stops and roars. When the player shoots, a ray is cast from the camera position to detect if it hits the dinosaur's collision volume. If hit, the hit time is accumulated, and the dinosaur dies when the time exceeds the threshold.

```
1 void TRexEntity::updateHitDetection(const Vec3&
  rayOrigin,
2   const Vec3& rayDirection, bool isShooting,
  float dt) {
3   collisionBox.updateHitTime(rayOrigin,
  rayDirection, isShooting, dt);
4 }
```

10 AABB Collision Detection and Response

The collision detection system is implemented in Collision.h and Collision.cpp, using the Axis-Aligned Bounding Box (AABB) algorithm. It supports point detection and ray intersection detection. Ray detection uses the Slab method to calculate the intersection point between a ray and an AABB.

```
1 bool CollisionBox::rayIntersect(const Vec3&
  rayOrigin,
2   const Vec3& rayDirection) const {
3   Vec3 min = center - halfSize;
4   Vec3 max = center + halfSize;
5
6   float tmin = 0.0f;
7   float tmax = 1000000.0f;
8
9   for (int i = 0; i < 3; i++) {
10    if (fabsf(rayDirection.coords[i]) <
11        0.0001f) {
12        if (rayOrigin.coords[i] < min.coords
13            [i] ||
14            rayOrigin.coords[i] > max.coords
15            [i])
16            return false;
17    } else {
18        float invD = 1.0f / rayDirection.
19        coords[i];
20        float t0 = (min.coords[i] -
21        rayOrigin.coords[i]) * invD;
22        float t1 = (max.coords[i] -
23        rayOrigin.coords[i]) * invD;
24        if (invD < 0.0f) std::swap(t0, t1);
25        tmin = t0 > tmin ? t0 : tmin;
26        tmax = t1 < tmax ? t1 : tmax;
27        if (tmax < tmin) return false;
28    }
29 }
30 return tmax > 0.0f;
31 }
```

11 Normal Mapping

Normal mapping is implemented in the PSNormalMapped.txt and PSAnimNormalMapped.txt shaders. Tangent space normal maps are used, converting normals from tangent space to world space through the TBN matrix to enhance surface details.

```
1 float3 normalMapSample = normalMap.Sample(
  samplerLinear, input.TexCoords).rgb;
2 if (length(normalMapSample) > 0.1) {
3   float3 tangentNormal = normalize(
4   normalMapSample * 2.0 - 1.0);
5   float3 T = normalize(input.Tangent);
6   float3 B = cross(N, T);
7   float3x3 TBN = float3x3(T, B, N);
8   worldNormal = mul(tangentNormal, TBN);
9   worldNormal = normalize(worldNormal);
10 }
```

12 Particle System Effects

The particle system is implemented in Particles.h and Particles.cpp for rendering falling leaf effects. Instanced rendering technology is used, where each particle is a Billboard that always faces the camera. Particle positions and animation offsets are stored in instance buffers and transformed through the vertex shader.

```
1 void Particles::init(Core* core, PSOManager*
  psos, Shaders* shaders,
2   int leafCount, Vec3 distributionVolume,
  float billboardSize) {
3   vertices.push_back(createStaticVertex(Vec3
4   (-0.5f, -0.5f, 0), ...));
5   vertices.push_back(createStaticVertex(Vec3
6   (0.5f, -0.5f, 0), ...));
7
8   for (int i = 0; i < numLeaves; i++) {
9     LEAF_INSTANCE_DATA leaf;
10    leaf.position.x = ((float)rand() /
11    RAND_MAX - 0.5f) * distributionVolume.x;
12    leaf.position.y = ((float)rand() /
13    RAND_MAX) * distributionVolume.y;
14    leaf.position.z = ((float)rand() /
15    RAND_MAX - 0.5f) * distributionVolume.z;
16    leaf.offset = (float)rand() / RAND_MAX *
17    fallHeight;
18    instanceData.push_back(leaf);
19 }
20 }
```

13 Conclusion

This project successfully implements all required features, constructing a fully functional 3D game engine. Through DirectX 12's API, an efficient rendering pipeline is achieved; through custom file formats, flexible resource management is realized; through modular code design, good extensibility is achieved. All features have been tested in practice and can run stably with excellent visual effects.